Walchand College of Engineering, Sangli

Department of Computer Science & Engineering

7CS353 — Machine Learning Lab

TY B.Tech CSE | Semester V | AY 2025-26

Assignment 10

Model Optimization Pipeline

Regularization · Feature Importance · Hyperparameter Tuning

Attribute	Detail
Linked Theory Course	7CS302 — Machine Learning
CO Mapping	CO4 (Design) = 10 marks | Total = 10 marks
Dataset	loan_approval.csv (500 applicants, 10 features, binary target: approved, seed = 353)
Prerequisite Assignments	A7 (Logistic Regression), A8 (Decision Tree / Random Forest)
Duration	1 lab session
Submission	Jupyter Notebook (.ipynb) + PDF

PART A — Tutorial & Reference

Guided theory and worked examples. Not independently graded — use it while completing Part B.

A1 The Problem: Bias, Variance, and Overfitting

Every supervised model makes two kinds of errors on unseen data:

Error type	Cause	Symptom	Fix
Bias (underfitting)	Model too simple to capture the pattern	High train error AND high test error	More features, more complex model
Variance (overfitting)	Model too complex, memorises training noise	Low train error, HIGH test error	Regularization, fewer features, more data

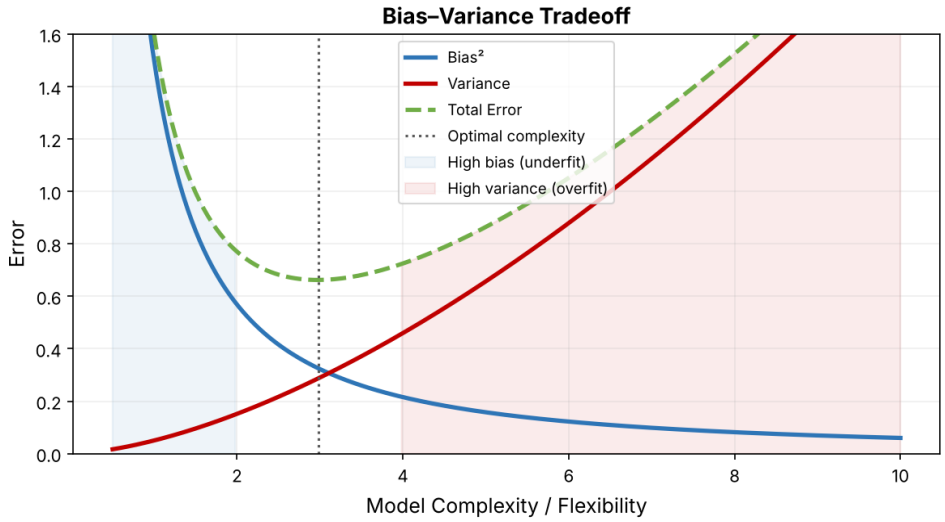


Figure 1: Bias–Variance tradeoff. The sweet spot minimises total error. Regularization moves a high-variance model left toward that sweet spot.

A2 Regularization

A2.1 What Regularization Does

Regularization adds a **penalty term** to the loss function that discourages large coefficients, limiting how much the model can fit noise:

$$\text{Regularized Loss} = \text{Data Loss} + \lambda \cdot \text{Penalty}(w)$$

λ (lambda or alpha) controls the strength. Higher λ → stronger penalty → simpler model → less overfitting, but potentially more bias.

A2.2 Ridge, Lasso and ElasticNet

Method	Penalty	Formula	Key behaviour
Ridge (L2)	Sum of squared coefficients	$\lambda \Sigma w_i^2$	Shrinks all coefficients toward zero; none exactly zero. Good when many features contribute a little.
Lasso (L1)	Sum of absolute coefficients	$\lambda \Sigma \lvert w_i \rvert$	Drives some coefficients to exactly zero → built-in feature selection. Good when few features truly matter.
ElasticNet	Mix of L1 + L2	$\lambda_1 \Sigma \lvert w_i \rvert + \lambda_2 \Sigma w_i^2$	Best of both: sparsity from L1, stability from L2. Controlled by l1_ratio.

In sklearn's LogisticRegression, the parameter is **C = 1/λ** — so *small C = strong regularization* and *large C = weak regularization*.

A2.3 The Regularization Path

A **regularization path** traces how each feature's coefficient changes as λ varies from very strong (all coefs ≈ 0) to very weak (unpenalised model). Reading it reveals:

- Which features are selected first as regularization weakens — these are the most important
- Which features stay at zero until λ is very small — these are weakly predictive or redundant
- The value of λ at which the model stabilises — a good candidate for your chosen regularization strength

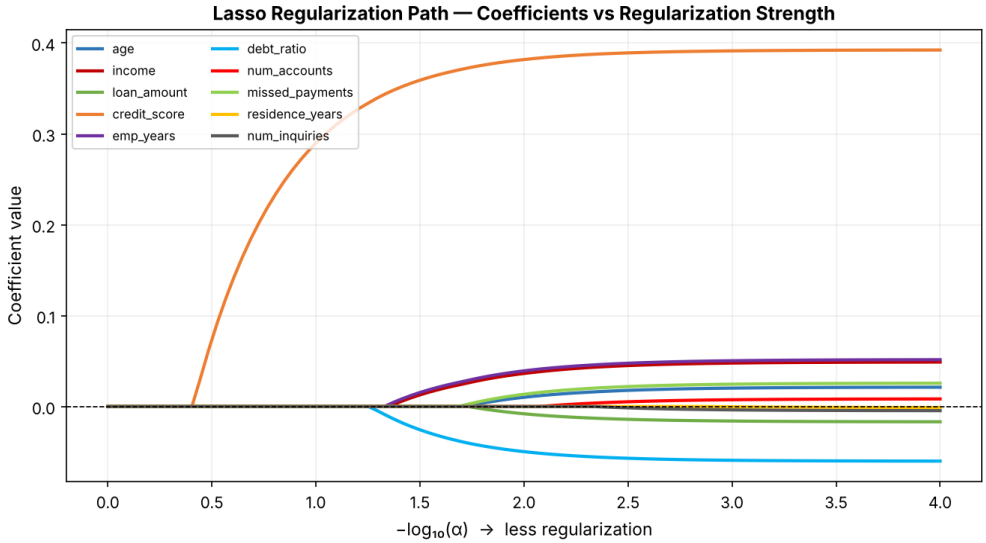


Figure 2: Lasso regularization path on loan_approval.csv. credit_score enters first (most important); the six noisy features stay near zero across the entire path.

A3 Feature Importance via Lasso Coefficients

After fitting a Lasso (L1) logistic regression, each surviving coefficient directly measures that feature's contribution — larger absolute value = stronger influence on the prediction. This gives a cheap, built-in ranking:

- Extract model.coef_[0] after fitting LogisticRegression(C=…, penalty="l1", solver="liblinear")
- Sort by absolute value — the top features drove the model's decisions
- Features with coefficient = 0 were regularized out and can be dropped
- Retrain on the reduced feature set and compare AUC to the full model

Important caveat: coefficient magnitude is only meaningful when features are *standardised* (zero mean, unit variance). Always apply StandardScaler before fitting.

A4 Hyperparameter Tuning with GridSearchCV

A4.1 Cross-Validation

A single train/test split can be a lucky or unlucky split. **k-fold cross-validation** gives a more reliable estimate: split data into k equal folds; train on k−1, validate on the remaining 1; repeat k times; average the scores.

CV setting	When to use
cv=5 (5-fold)	Default; good balance of bias and variance in the estimate
cv=10 (10-fold)	More reliable estimate; ~2× slower
StratifiedKFold	Classification with class imbalance — preserves class ratio in each fold

A4.2 GridSearchCV

GridSearchCV exhaustively tries every combination of hyperparameter values you specify, evaluates each via cross-validation, and returns the best combination:

```
param_grid = {'C': [0.001, 0.01, 0.1, 1, 10, 100]} grid =
GridSearchCV(LogisticRegression(...), param_grid, cv=5, scoring='roc_auc')
grid.fit(X_train, y_train) print(grid.best_params_, grid.best_score_)
```

After fitting, always evaluate **grid.best_estimator_** on the *held-out test set* — never the validation fold used during CV.

PART B — Independent Tasks (10 marks)

Dataset: loan_approval.csv (500 applicants · 10 features · binary target: approved · seed = 353)

Answer every interpretation prompt in a Markdown cell below the code.

B1 Load Data and Establish a Baseline

- Load the dataset. Perform required preprocessing.
- Fit a default LogisticRegression(C=1.0, max_iter=1000) on the training set. Report: Accuracy, F1-score, ROC-AUC on the test set. This is your baseline.
- In a Markdown cell: what does an AUC of 0.5 mean? What does 1.0 mean? What does your baseline AUC tell you?

B2 Regularization — Ridge, Lasso, and ElasticNet

B2.1 Fit three models on the training set: LogisticRegression with (a) penalty="l2" (Ridge), (b) penalty="l1", solver="liblinear" (Lasso), (c) penalty="elasticnet", solver="saga", l1_ratio=0.5. Use C=0.1 for all. Report test AUC for each.

B2.2 Lasso regularization path: loop C over [0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1, 5, 10]. For each C, fit Lasso and record the 10 coefficients. Plot coefficient value vs log(C) — one line per feature. Which features reach zero first as C decreases?

B2.3 Print a table: Feature | Ridge coef (C=0.1) | Lasso coef (C=0.1). How many features does Lasso zero out at C=0.1?

B2.4 Markdown: explain in 3 sentences why Ridge keeps all features while Lasso can zero some out. Relate your answer to the geometry of L1 vs L2 penalties.

B3 Feature Importance and Hyperparameter Tuning

Use Lasso coefficients as your feature importance measure throughout this task.

B3.1 Fit Lasso (C=0.1). Extract and sort features by |coefficient|. Plot a horizontal bar chart: feature names on y-axis, |coef| on x-axis. Colour zero-coefficient features differently.

B3.2 Identify which features have coef = 0. Drop them from X_scaled. Call this X_reduced. Retrain the baseline LogisticRegression(C=1.0) on X_reduced. Report test AUC. Did dropping weak features hurt performance?

B3.3 GridSearchCV: on X_reduced, search C over [0.001, 0.01, 0.1, 1, 10, 100] with cv=5, scoring="roc_auc". Print best_params_ and best_score_.

B3.4 Evaluate grid.best_estimator_ on the test set. Fill in this table with your results:

Model	Features used	Test AUC
Baseline (C=1, all features)	10	???
Lasso-reduced (C=1)	???	???
Tuned (best C, reduced)	???	???

B4 Design Task — Real-World Adaptation (Take-Home)

Business Constraint
Your bank's compliance team issues the following requirement:
"The loan approval model must use at most 5 features, must be explainable to auditors in plain language, and must achieve at least 90% of the baseline AUC."

B4.1 Select exactly 5 features. Justify your choice: why these 5? Use evidence from B2 (regularization path) and B3 (coefficient ranking).

B4.2 Tune the regularization strength for your 5-feature model using GridSearchCV (same grid as B3.3). Report best C and test AUC.

B4.3 Compute the AUC ratio: your 5-feature model AUC / baseline AUC × 100%. Does it meet the ≥90% constraint?

B4.4 Write a short design report (≥5 sentences in a Markdown cell) covering: (a) which features you selected and why, (b) which regularization method you used and why, (c) what you would do if the 90% constraint was not met, (d) one potential fairness concern with this model in a real bank.

Assessment Rubric

CO	Weight	Rubric Statement	Very Good (90–100%)	Good (75–89%)	Average (50–74%)	Needs Improvement (<50%)	Not Done (0%)
CO4	**10**	Design an optimized, real-world-adapted ML pipeline: apply regularization, select features via Lasso, and	Pipeline complete; regularization + feature selection +	All three steps present; justification partial or	Two steps present; tuning or feature selection	Only one step; no justification.	Not attempted.

CO	Weight	Rubric Statement	Very Good (90–100%)	Good (75–89%)	Average (50–74%)	Needs Improvement (<50%)	Not Done (0%)
		tune hyperparameters with evidence-based justification.	tuning all present; justification clear and specific with metric evidence.	missing one step.	missing; vague justification.		

Performance Level	Marks Awarded
100% (full marks)	10
90%	9
75%	7–8
60%	6
50%	5
Below 50%	Proportional